Convolutional Neuronal Networks 2

Grundlage dieser Übung bildet das *Cats and Dogs*-Dataset:

`The training archive contains 25,000 images of dogs and cats. Train your algorithm on these files and predict the labels for test1.zip (1 = dog, 0 = cat).`

Das Dataset besteht aus zwei Archiven; wobei das Archiv `train.zip` die Trainings- und das Archiv `test1.zip` die Testdaten enthalten. Abbildung 1 zeigt den Inhalt des train-Archives. Dieser ist zu entnehmen, dass der Aufbau der Dateinamen einem bestimmten Muster folgt, und zwar: `{cat,dog}.<Lfd. Nummer>.jpg`.

```
$ tree train
train
├── cat.0.jpg
├── cat.1.jpg
├── cat.10.jpg
├── cat.100.jpg
├── cat.1000.jpg
├── cat.10000.jpg
├── cat.10001.jpg
├── cat.10002.jpg
├── cat.10003.jpg
├── cat.10004.jpg
├── cat.10005.jpg
```

Abbildung 1: Inhalt des train-Archives (Auszug)

Hintergrund ist, dass der Dateiname gleichzeitig als Label, nämlich um welches Tier es sich handelt, dient. Die fortlaufende Nummer (ID) ermöglich die eindeutige Zuorndung auf Code-Ebene. Der Inhalt des Archives `test1` folgt dem gleichen Muster. Nachdem wir Supervised Learning implementieren, sind die Daten so aufzubereiten, dass die hierfür typische Form „Beispiele und Antworten“ gegeben ist. Das Ergebnis dieses Arbeitsschrittes könnte bspw. ein *Data Frame* sein, dessen Inhalt sich wie folgt gestaltet:

	filename	category
0	cat.0.jpg	cat
1	cat.1.jpg	cat
2	cat.10.jpg	cat
3	cat.100.jpg	cat
4	cat.1000.jpg	cat

Abbildung 2: Date Frame mit Bezug auf Trainingsdaten

Die Spalte `filename` (=Beispiel) beinhaltet den umfassenden Dateinamen, die Spalte `category` (=Antwort) die Information, ob es sich um eine Katze oder um einen Hund handelt.

✐ 02.1 Laden Sie die erforderlichen Ressourcen `train.zip`[1] und `test.zip`[2], entpacken Sie diese im Projekt-Verzeichnis. Implementieren Sie jenen Code, den es braucht, um die Ausgabe in Abbildung 2 zu erzielen.

Zur Erinnerung: Ziel ist die Aufbereitung der Trainingsdaten. Demnach bildet das Archiv `train.zip` die Arbeitsgrundlage!

✐ 02.2 Zeigen Sie, dass 25.000 Trainingsbilder vorliegen und überprüfen Sie, ob das Dataset ausgewogen ist.

✐ 02.3 Implementieren Sie eine Funktion, die in Abhängigkeit der Argumente das gewünschte Bild anzeigt. Um ein Bild anzuzeigen, sind zwei logische Schritte erforderlich:

1. Bild laden: Hierzu ist die Keras-Funktion `load_image`[3] überaus hilfreich.
2. Bild anzeigen: Zum Anzeigen eines Bildes benötigen wir die bereits bekannte `matplotlib.pyplot`-Funktion `imshow`.

Bei Betrachtung der Daten kommt man zu dem Schluss, dass mit den Klassen *Cat* und *Dog* eine Binärklassifikation vorliegt. Labels: 0 = cat, 1 = dog

✐ 02.4 Entwickeln und kompilieren Sie ein Modell, das nachfolgende Architektur aufweist:

```
Conv2D (16 Filter, 3x3 Kernel)
MaxPooling (2x2)

Conv2D (32 Filter, 3x3 Kernel)
MaxPooling (2x2)

Conv2D (64 Filter, 3x3 Kernel)
MaxPooling (2x2)

GlobalAveragePooling2D # ersetzt Flatten
Dense(1)
```

Aktivierungsfunktionen sowie Verlustfunktion und Optimizer gilt es gemäß Aufgabencharakteristik zu wählen.

GlobalAveragePooling2D ersetzt Flatten vollständig, da die räumlichen Dimensionen bereits auf einen einzigen Wert pro Kanal reduziert werden. Das Ergebnis ist ein eindimensionaler Feature-Vektor ohne weitere Flatten-Operation.

[1] http://htl.boxtree.at/lehre/media/docs/ki/catsanddogs/train.zip
[2] http://htl.boxtree.at/lehre/media/docs/ki/catsanddogs/test1.zip
[3] https://keras.io/api/preprocessing/image/#loadimg-function

Datenaufbereitung

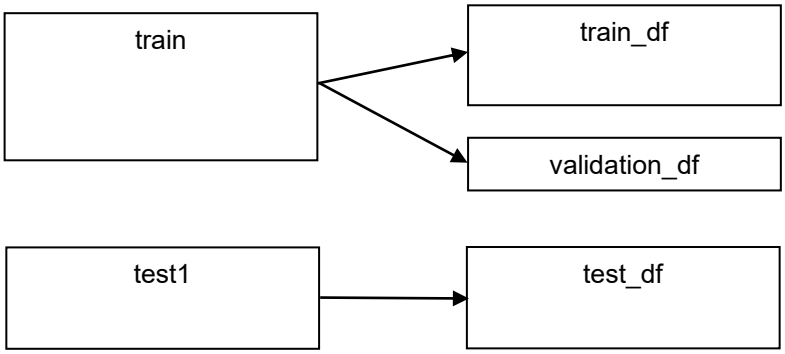
Bevor es an das Trainieren des Modells geht, gilt es noch die Daten aufzubereiten. Dem geübten Schüler/der geübten Schülerin ist sicherlich nicht entgangen, dass das in Aufgabe 02.1 erstellte Data Frame nur Referenzen auf die Dateien, aber keine Pixelinformationen enthält.

Im Prinzip braucht es folgende Schritte:

1. Bilddaten einlesen.
2. JPEG-Dateien in RGB-Pixelwerte umwandeln.
3. RGB-Pixelwerte in Fließkommazahltensoren umwandeln.
4. Normalisierung der Pixelwerte in das Intervall [0, 1].

Das mag mühsam erscheinen, was es in der Tat auch ist. Aber glücklicherweise verfügt Keras über einige Klassen, die diese Aufgaben für uns erledigen können. Zu diesem Zweck gibt es mit `tf.keras.preprocessing.image` ein Modul zur Bildverarbeitung. Es enthält insbesondere die Klasse `ImageDataGenerator`, die es ermöglicht, auf der Festplatte gespeicherte Bilddaten in einen Stapel von Tensoren umzuwandeln.

Vorbereitung der Data Frames:



- `train_df` und `validation_df` bilden die Grundlage für das Training.
- Das Testen des trainierten Modells erfolgt dann mit `test_df`.

Die praktische Umsetzung dieses Schrittest gestaltet sich folgendermaßen, wobei `data` das Data Frame aus 02.1 ist:

```
# df["category"] = df["category"].map({0: 'cat', 1: 'dog'})

train_df, validation_df = train_test_split(data, test_size=0.20,
stratify=df["category"], random_state=12)

train_df = train_df.reset_index(drop=True)
validation_df = validation_df.reset_index(drop=True)
```

```
print(train_df.shape)
print(validation_df.shape)
```

Output:

```
(20000, 2)
(5000, 2)
```

ImageDataGenerator

Ein `ImageDataGenerator` lädt Bilder während des Trainings direkt von der Festplatte, verarbeitet sie in handlichen Batches und führt dabei automatisch die gewünschte Vorverarbeitung (Normalisierung, Augmentierung) durch. Dadurch müssen nicht alle Bilder gleichzeitig in den Arbeitsspeicher geladen werden, was das Training wesentlich speicherschonender macht.

Erstellung der Generatoren:

```
train_datagen = ImageDataGenerator(rescale=1./255) ❶
validation_datagen = ImageDataGenerator(rescale=1./255) ❶

train_generator = train_datagen.flow_from_dataframe( ❷
    dataframe = train_df, ❸
    directory = "train", ❹
    x_col = "filename", ❺
    y_col = "category", ❻
    target_size = (150,150), ❼
    batch_size = 64, ❽
    class_mode = "binary") ❾

validation_generator = validation_datagen.flow_from_dataframe(
    dataframe = validation_df,
    directory = "train",
    x_col = "filename",
    y_col = "category",
    target_size = (150,150),
    batch_size = 64,
    class_mode = "binary")
```

Erläuterung:

1. Skalieren der Bilder mit dem Faktor 1/255 (Details siehe API[4])

[4] https://keras.io/api/preprocessing/image/#imagedatagenerator-class

2. Ausgangsbasis bilden Dataframes, demnach ist die entsprechende Methode zu verwenden. Es gibt Alternativen, wie `flow_from_directory`.
3. Dataframe mit den Referenzen auf die Bilddaten.
4. Das Verzeichnis, in welchem die Bilddaten zu finden sind.
5. Spalten des Dataframes, die die Details bzgl. „Beispiele“ enthält.
6. Spalte des Dataframes, die die Details bzgl. „Antworten“ enthält.
7. Durch proportionale Verkleinerung wird sichergestellt, dass alle Bilder dieselbe Größe haben. Die Eingaben sind somit konsistent. Erfolgt beim Einlesen der Bilder.
8. Der Generator liest während des Trainings jeweils 64 Bilder, die dann dem Modell übergeben werden. Es gilt einen Kompromiss zw. Effizienz und Ressourcenverbrauch zu erzielen.
9. Der Aufgabenstellung nach ist „binary“ zu wählen.

Hinweis: Was ein Python-Generator im Detail leistet und unter welchen Umständen deren Verwendung Sinn macht, ist dieser Quelle[5] zu entnehmen. Fürs Erste reicht es zu wissen, dass Generatoren iterativ Daten generieren, was vor allem bei großen Datenmengen Sinn macht. Im Gegensatz zu regulären Funktionen, die einen Wert zurückgeben und dann beendet werden, können Generatoren eine Sequenz von Werten generieren, ohne den „gesamten“ Speicher auf einmal zu belegen.

```
for data_batch, labels_batch in train_generator:
    print(data_batch.shape)
    print(labels_batch.shape)
    break
```

Output:

```
(64, 150, 150, 3)
(64, )
```

Betrachtet man die Ausgaben, ist festzustellen, dass der Generator Stapel von 150 x 150 Pixel große RGB-Bilder sowie den Antworten-Vektor liefert. Jeder dieser Stapel enthält 64 Samples, was auf die `batch_size` des Generators zurückzuführen ist. Die break-Anweisung braucht es, da der Generator das Zielverzeichnis durchläuft.

Jetzt ist es an der Zeit, das Modell zu trainieren. Hierzu gibt es bzw. gab es mehrere `fit`-Varianten. Denn laut Keras API ist die für Generatoren vorgesehene `fit_generator`-Methode *deprecated*. Es wird auf die Verwendung von `fit` – auch bei Generatoren – hingewiesen.

```
history = model.fit(
      train_generator, ❶
      steps_per_epoch = 312, ❷
      validation_data = validation_generator, ❸
```

[5] https://www.guru99.com/python-yield-return-generator.html

```
epochs=3) ❹
```

1. Das erste Argument ist der `train_generator`, der sowohl die Beispiele als auch Antworten enthält.
2. Wir verwenden 20.000 Bilder für das Training. Bei der Erstellung des Trainings-Generators ist als `batch_size` 64 angeführt. Daraus folgt: 20.000/64=312 (gerundet). Das bedeutet, dass während jeder Epoche 312 * 64 = 19968 Bilder aus dem Trainingsgenerator gezogen werden. Möchte man mit weniger Bilder trainieren, kann `man steps_per_epoch` reduzieren.
3. Nach jedem Schritt `steps_per_epoch` soll eine „Zwischen-Validierung“ vorgenommen werden.
4. 3 Epochen sind nicht gerade üppig. Optimal wären höchstwahrscheinlich 30 bis 40 Epochen. Bei 3 Epochen macht die Visualisierung der Daten in `history` wenig Sinn.

Wie gut ist nun das Modell bei den unbekannten Testdaten?

Aufbereitung der Testdaten für predict

Dataframe erstellen:

```
test_filenames = os.listdir("test1")
test_df = pd.DataFrame({
    'filename': test_filenames
})
test_df.shape[0]
```

Output:

```
12500
```

Den Test-Generator erstellen:

```
test_gen = ImageDataGenerator(rescale=1./255)
test_generator = test_gen.flow_from_dataframe(
    test_df,
    directory="test1",
    x_col='filename',
    y_col=None,
    class_mode=None,
    target_size=(150,150),
    batch_size=64,
    shuffle=False
```

```
)
```

Output:

```
Found 12500 validated image filenames.
```

Die Antworten (y_col) braucht es in diesem Fall nicht. Diese generiert uns ja `predict`:

```
test_generator.reset()
pred=model.predict(test_generator, verbose=1)
```

Das Ergebnis:

```
pred
```

Output:

```
array([[0.5213046 ],
       [0.47121802],
       [0.5362638 ],
       ...,
       [0.5831151 ],
       [0.5206357 ],
       [0.5702343 ]], dtype=float32)
```

Der Output ist wie folgt zu verstehen: Eintrag 0 ist wohl ein Hund, da 0,5213046 größer als 0,5 ist. Eintrag 1 wurde als Katze vorhergesagt, Eintrag 2 als Hund usw.

Die Vorhersagen in die definierten Kategorien überführen und dem test_df hinzufügen:

```
pred_rounded = [1 if val > 0.5 else 0 for val in pred]
test_df["category"] = pred_rounded
test_df.head(5)
```

Output:

	filename	category
0	1.jpg	1
1	10.jpg	0
2	100.jpg	1
3	1000.jpg	1
4	10000.jpg	1

Mit ein wenig Code könnte man nun herausfinden, wo unser Modell richtig bzw. falsch lag (vgl. MNIST-Bsp.). Wir sind jedoch an konkreten Beispielen interessiert:

```
sample_test = test_df.head(18)
plt.figure(figsize=(12, 24))
for index, row in sample_test.iterrows():
```

```
    filename = row['filename']
    category = row['category']
    img = load_img("test1/"+filename, target_size=(150,150))
    plt.subplot(6, 3, index+1)
    plt.imshow(img)
    plt.xlabel(filename + '(' + "{}".format(category) + ')' )
plt.tight_layout()
plt.show()
```

Output:

1.jpg(1)
10.jpg(0)
100.jpg(1)
1000.jpg(1)
10000.jpg(1)
10001.jpg(0)
10002.jpg(1)
10003.jpg(1)
10004.jpg(1)
10005.jpg(1)
10006.jpg(0)
10007.jpg(1)
10008.jpg(1)
10009.jpg(1)
1001.jpg(1)
10010.jpg(1)
Sport
10011.jpg(1)
10012.jpg(0)

Zur Erinnerung:

```
train_generator.class_indices.items()
```

Output:

```
dict_items([('cat', 0), ('dog', 1)])
```

✐ 02.5 Bereiten Sie die Daten auf, trainieren Sie das Modell und führen Sie Vorhersagen (predict) mit den Test-Daten (Ordner test1) durch.